

Robótica
Informe capítulo 7– RVC v2 Peter Corke

Cinemática de brazos robóticos

Docente: Jaime Enrique Arango Castro

Estudiante:
Cristhian Mateo Almeida Gómez

Universidad Nacional de Colombia
Sede Manizales



Resumen

La cinemática es la rama de la robótica que estudia el movimiento de los robots sin considerar fuerzas ni masas, enfocándose en la determinación de las posiciones, velocidades y aceleraciones de las partes del robot. El concepto de **grados de libertad (DoF)** es clave, ya que se refiere a los movimientos independientes que puede realizar un robot. Los robots más comunes en robótica, como los **SCARA** (4 DoF) y los robots de **eslabones en paralelo** tienen una estructura que permite una amplia gama de movimientos.

Grados de libertad DoF

Los DoF hacen referencia a los parámetros que un robot puede mover de forma independiente. Un brazo humano, por ejemplo, tiene 7 DoF, mientras que robots industriales como el **SCARA** cuentan con 4 DoF para movimientos más precisos y específicos.

SCARA: 4 DoF

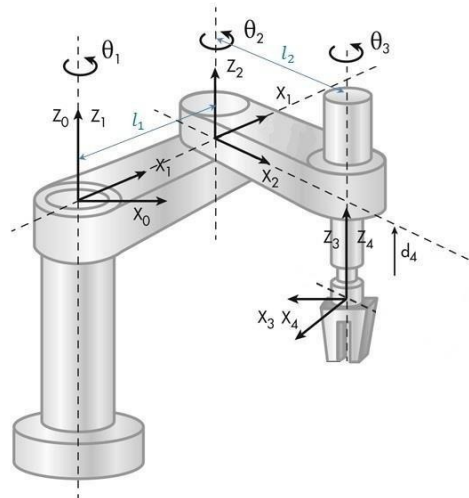


Figura 1.1 4 DoF.

Eslabones en Serie: Eslabones uno tras otro, flexibles con gran área de trabajo.

Eslabones en Paralelo: Sí manejan poca masa presentan gran aceleración, rígidos y difícil de modelar.

SCARA (Selective Compliance Assembly Robot Arm):

En la imagen izquierda se muestra un robot SCARA, que tiene 4 DoF:

- 2 rotaciones horizontales (θ_1, θ_2)
- 1 movimiento vertical (Z)
- 1 rotación en el eje Z (θ_4)

Este tipo de robot es muy utilizado en ensamblaje industrial porque es rápido y preciso en movimientos horizontales. Tiene configuración en serie, es decir, un eslabón se conecta tras otro, como un brazo humano.

Delta:

En la imagen derecha se muestra un robot Delta, compuesto por tres brazos unidos a una plataforma móvil. Es un robot paralelo, lo que le permite mover el efector final con alta aceleración y precisión. Se usa en tareas de pick-and-place de alta velocidad (por ejemplo, en empaques de alimentos o electrónicos). Tiene una estructura rígida y liviana, lo que permite movimientos muy rápidos.



Figura 1.2.Delta

Eslabones en Serie: Son eslabones conectados en secuencia, flexibles y con un amplio rango de movimiento.

Eslabones en Paralelo: Estos eslabones son más rígidos y difíciles de modelar, pero al manejar poca masa, presentan una gran aceleración.

SCARA (Selective Compliance Assembly Robot Arm): El robot SCARA, mostrado en la figura 1.1, tiene 4 grados de libertad (DoF):

- 2 rotaciones horizontales (θ_1 , θ_2)
- 1 movimiento vertical (Z)
- Este robot se utiliza ampliamente en aplicaciones industriales de ensamblaje debido a su alta velocidad y precisión en movimientos horizontales, además presenta 1 rotación en el eje Z θ_4 . Su configuración en serie, donde un eslabón se conecta a otro, lo hace similar a un brazo humano.

Delta:

El robot Delta, ilustrado en la figura 1.2, está compuesto por tres brazos que se unen a una plataforma móvil. Como robot paralelo, permite un movimiento del efector final con alta aceleración y precisión. Es ideal para tareas rápidas de pick-and-place, como en el empaquetado de alimentos o productos electrónicos. Su estructura es rígida y liviana, lo que le otorga gran capacidad de movimiento rápido.

7.1.1 Cinemática Directa 2D.

En la cinemática directa se determina la posición y orientación del efector final (la herramienta o punta del robot) a partir de los valores de sus articulaciones. En este caso, se está trabajando en 2D, es decir, sobre un plano.

Caso 1: Rotación + Traslación con una articulación (R)

$${}^0\xi_E = \xi^r(q) \oplus \xi^{t_x}(a_1)$$

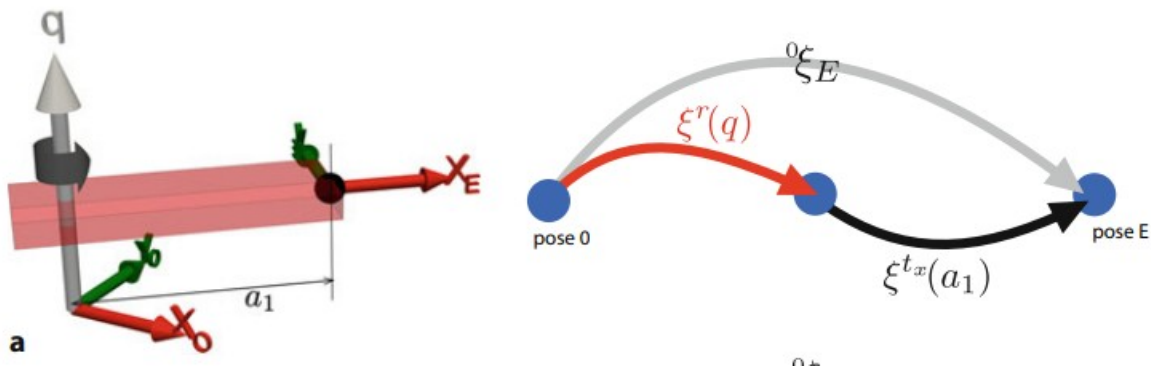
donde:

${}^0\xi_E$: Representa la posición final (posición y orientación) del efector respecto al sistema de referencia base (0).

$\xi^r(q)$: Es una rotación variable, determinada por el ángulo q , que depende de la articulación del robot (por ejemplo, una articulación rotacional).

$\xi^{t_x}(a_1)$: Es una traslación constante sobre el eje X, con longitud a_1 (una distancia fija entre eslabones).

$\oplus$: Representa la composición de transformaciones, es decir, se aplica primero una rotación y luego una traslación.



Esta formulación describe cómo un robot, con una sola articulación, realiza un movimiento rotacional seguido de una traslación lineal en un plano para alcanzar la posición deseada de su efector.

```
a1 = 1;
e = ET2.R() * ET2.tx(a1);
[2] ✓ 0.0s

len(e)
[3] ✓ 0.0s
... 2

e
[4] ✓ 0.0s
... R(q) ⊕ tx(1)
...

e.fkine(pi / 6)
[5] ✓ 0.0s
...
0.866 -0.5 0.866
0.5 0.866 0.5
0 0 1

SE2.Rot(pi / 6) * SE2.Tx(a1)
[6] ✓ 0.0s
...
0.866 -0.5 0.866
0.5 0.866 0.5
0 0 1
```

Para implementar, se emplea la librería Robotics Toolbox para Python, que permite definir transformaciones homogéneas en el espacio. En el código, se construye una cadena cinemática que combina una rotación ($ET2.R()$) con una traslación ($ET2.tx(a1)$), siendo $a1 = 1$). Al aplicar una rotación de $\pi/6$ radianes (30°), se calcula la posición final del efector mediante el método `fkine`, que evalúa la transformación completa.

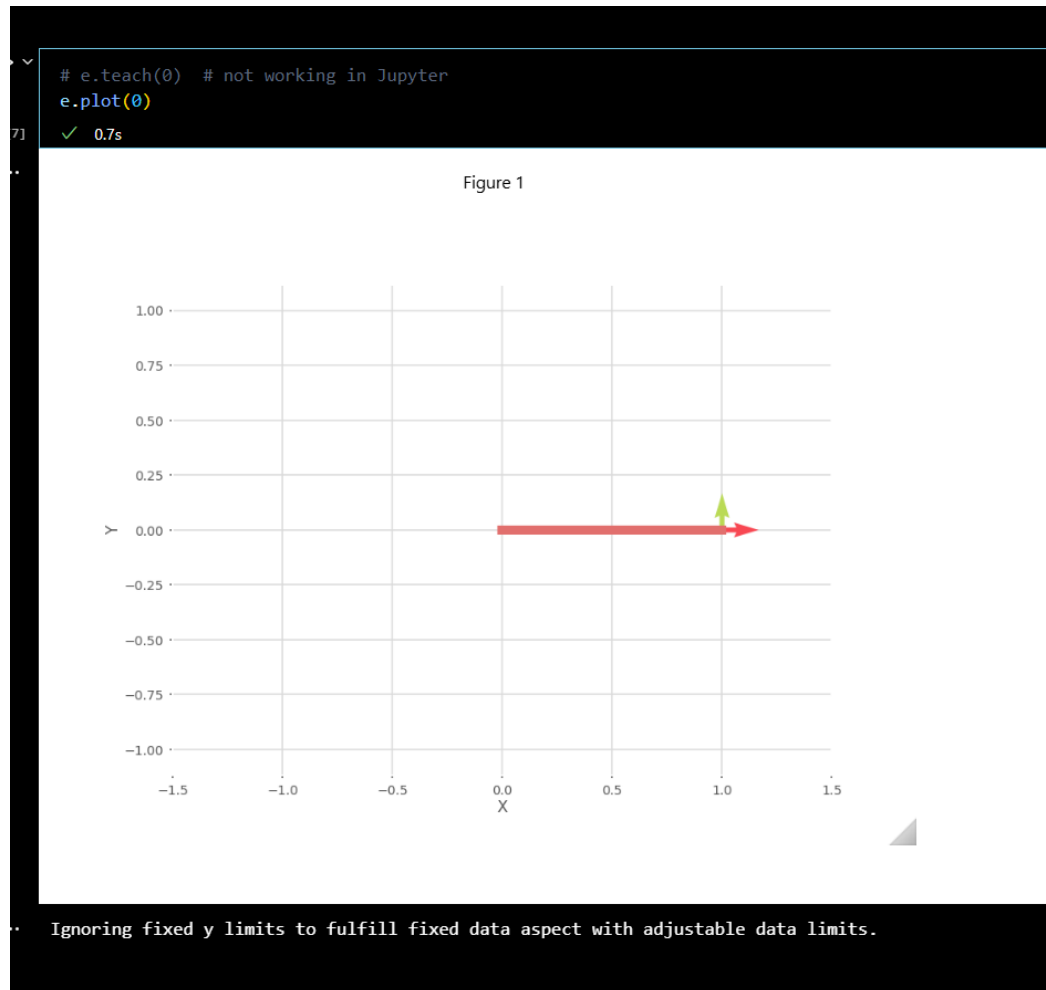
Implementación y Resultado

El resultado se expresa como una matriz homogénea 3x3, la cual contiene la información tanto de rotación como de traslación. Esta matriz permite ubicar con precisión la nueva posición y orientación del efector en el plano. Además, se visualiza gráficamente el resultado, mostrando cómo el efector se desplaza y rota respecto a su origen. Además, se muestra la equivalencia entre la notación simbólica y su evaluación numérica, confirmando la validez del modelo.

Para implementar la cinemática directa en un entorno de programación, se utiliza la librería **Robotics Toolbox** en Python. En el código, se construye una cadena cinemática combinando una rotación `ET2.Rot` y una traslación (`ET2.Tx`).

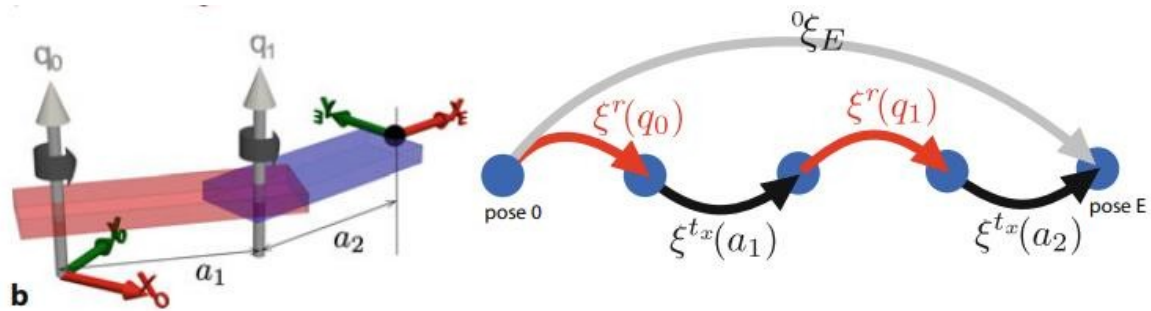
Este método se utiliza para evaluar la transformación de la posición final del efector. A través del código, se puede aplicar una rotación de 30° y calcular la posición final del efector en función de las entradas de las articulaciones.

El resultado se obtiene mediante una matriz homogénea de 3×3 que contiene tanto la rotación como la traslación. Esto permite obtener la posición y orientación final del efector en un sistema de coordenadas 2D, y también muestra cómo el efector se mueve respecto a su origen, validando la transformación.



Esta funcionalidad es útil para comprender mejor el comportamiento dinámico del sistema y cómo afectan las transformaciones individuales a la posición final del efector.

Caso 2: Rotación + Traslación con dos articulaciones (RR)



El robot tipo **RR** se compone de dos eslabones conectados por articulaciones rotacionales. Esta estructura permite que el efector final (punta del brazo) se desplace en diversas posiciones y orientaciones dentro del plano 2D, lo que es fundamental para tareas de manipulación en ese espacio.

El modelo del robot se construye aplicando una secuencia de transformaciones:

1. **Rotación variable** (q_0): Esta transformación define el ángulo de la primera articulación.
2. **Traslación** (a_1): Representa la longitud del primer eslabón del robot.
3. **Rotación** (q_1): Corresponde al ángulo de la segunda articulación.
4. **Traslación** (a_2): Representa la longitud del segundo eslabón.

En este caso específico, las longitudes de los eslabones (a_1 y a_2) son de 1 unidad cada una, y los ángulos de las articulaciones son 30° y 40° respectivamente.

```
[8] a1 = 1; a2 = 1;
    e = ET2.R() * ET2.tx(a1) * ET2.R() * ET2.tx(a2);
    ✓ 0.0s

[9] e
    ✓ 0.0s
... R(q0) ⊕ tx(1) ⊕ R(q1) ⊕ tx(1)
...

[10] e.fkine(np.deg2rad([30, 40])).printline()
    ✓ 0.0s
... t = 1.21, 1.44; 70°
```

```
▷ T = SE2.Rot(np.deg2rad(30)) * SE2.Tx(a1) * SE2.Rot(np.deg2rad(40)) * SE2.Tx(a2);
  T.printline()
[11] ✓ 0.0s
... t = 1.21, 1.44; 70°
```

Estas transformaciones se combinan utilizando **matrices homogéneas**, lo que permite calcular la **pose final** del efector. El resultado de este cálculo se presenta en forma de una matriz de transformación que describe tanto la **posición** como la **orientación** del efector respecto al marco de referencia

original. En este caso particular, la posición final del efector es aproximadamente (1.21, 1.44) y la orientación total es de 70° .

```
e.n
e.joints()
✓ 0.0s

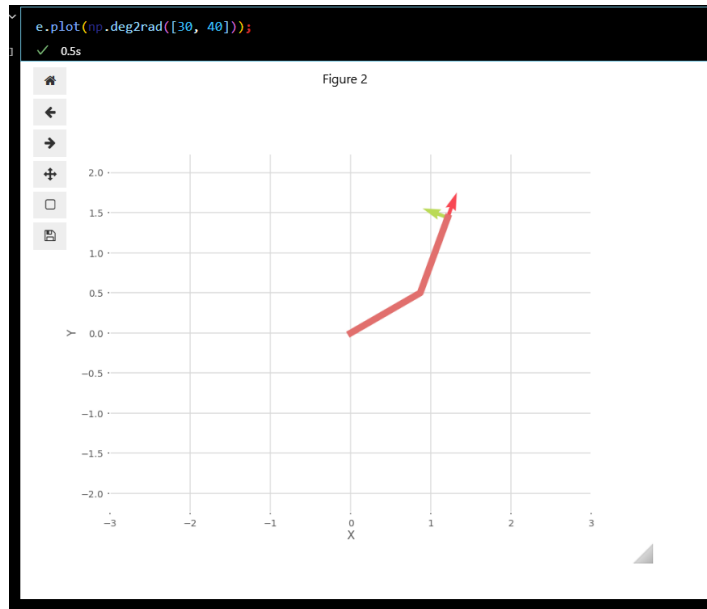
[R(q0), R(q1)]

e.structure
✓ 0.0s

'RR'
```

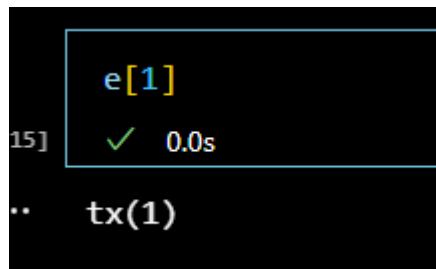
Este caso demuestra cómo se pueden encadenar múltiples transformaciones para modelar un robot más complejo, con un mayor número de grados de libertad y, por lo tanto, mayores posibilidades de movimiento, en comparación con el modelo básico del **Caso 1**.

Número de articulaciones (variables/ DoF) y cuales son:



Métodos del objeto e para ir por transformación:

El modelo cinemático del robot se puede gestionar usando índices en una lista de transformaciones. Cada índice en esta lista representa una rotación o una traslación:



Se accede a cada etapa del modelo cinemático del robot usando índices ($e[0]$, $e[1]$, etc.), donde cada una representa una rotación o traslación. En este caso, se tiene una secuencia tipo RR con dos rotaciones y dos traslaciones constantes.

Da la constante y su transformación:

```

e[1].eta
e[1].A()
✓ 0.0s

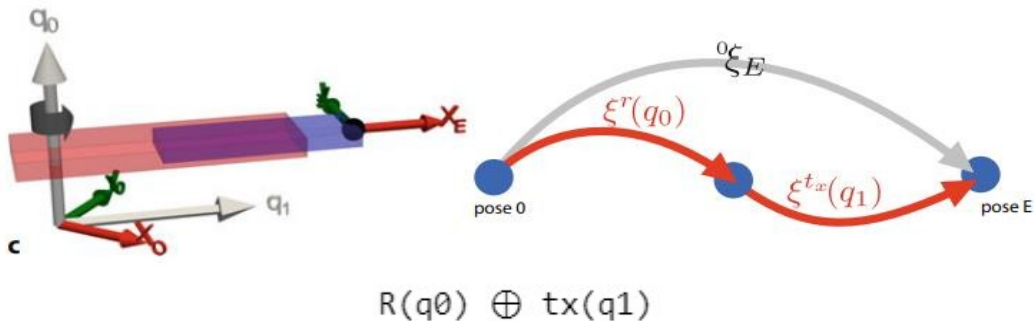
array([[ 1,  0,  1],
       [ 0,  1,  0],
       [ 0,  0,  1]])

```

- El primer índice corresponde a la rotación de la primera articulación.
- El segundo índice corresponde a la traslación del primer eslabón.
- El tercer índice corresponde a la rotación de la segunda articulación.
- El cuarto índice corresponde a la traslación del segundo eslabón.
-

Cada una de estas transformaciones se combina para determinar la posición final del efector. En resumen, el robot tipo **RR** emplea una secuencia de rotaciones y traslaciones para calcular la posición final del efector en el espacio 2D, lo que le otorga flexibilidad para realizar una variedad de movimientos.

Caso 3: Cinemática con articulaciones rotacional y prismática (RP)



Este estudio examina un robot de configuración plana con dos grados de libertad articulados:

- Una articulación rotacional ($R(q_0)$), que permite al primer eslabón girar respecto al origen.
- Una articulación prismática ($tx(q_1)$), que desplaza el segundo eslabón a lo largo del eje x , es decir, extiende o retrae el efector final a lo largo de una dirección recta.

```
[17] e = ET2.R() * ET2.tx(qlim=[1,2])
      ✓ 0.0s
...  R(q0) ⊕ tx(q1)
...
[18] e.structure
      ✓ 0.0s
...  'RP'
```

El modelo cinemático establece un rango operativo de 1 a 2 unidades para el movimiento prismático. La consulta a `e.structure` devuelve 'RP', validando la secuencia rotacional-prismática de la arquitectura robótica.

El propósito de este análisis es evaluar cómo los ajustes articulares afectan la pose del efector (posición y orientación), con especial atención a los límites de desplazamiento. Esta metodología ayuda a comprender la relación entre configuraciones articulares y capacidades de movimiento

7.1.2 Cinemática Directa como una cadena de eslabones

- **Eslabón:** Componente rígido con propiedades físicas (masa, inercia) que conecta articulaciones.
- **Articulación:** Mecanismo que habilita el movimiento relativo entre eslabones.

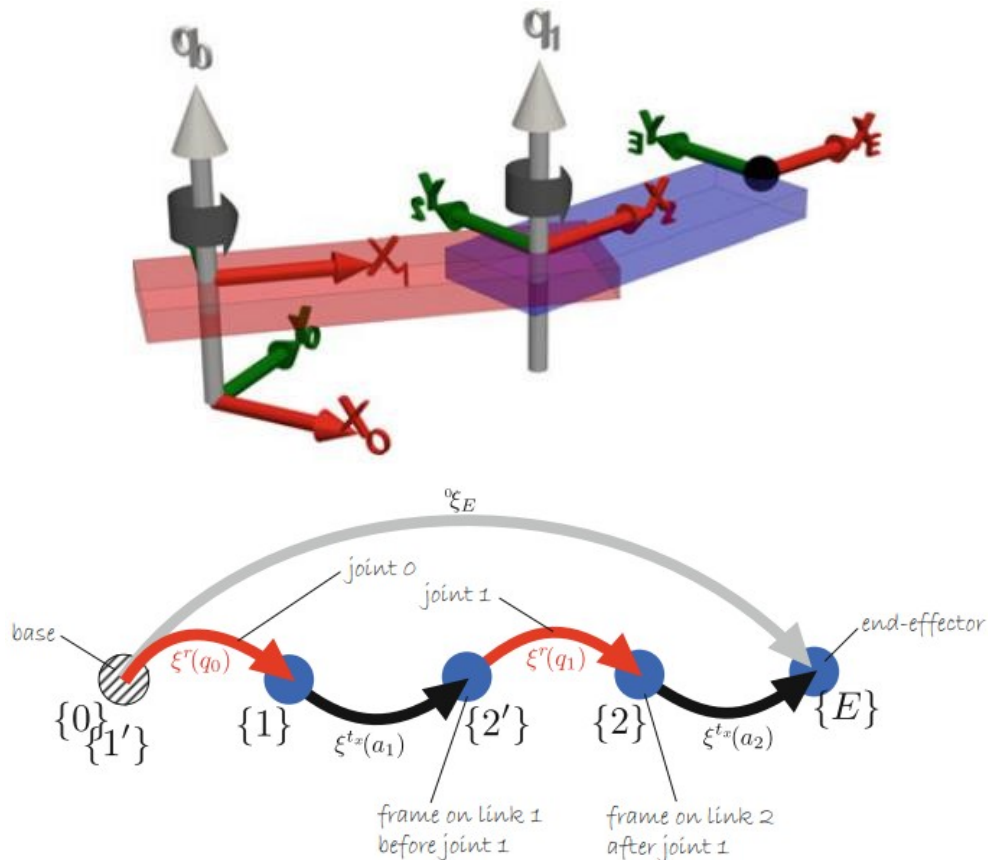
Jerarquía estructural:

- El eslabón base carece de predecesor.
- El eslabón efector no tiene sucesores.

Link/Eslabón: Parte física del robot, que posee masa, inercia y potencial de colisión. Estructura rígida entre las articulaciones.

Joins/Articulación: Elemento que permite el movimiento entre eslabones.

El eslabón de la base no tiene eslabón padre. El eslabón del efector no tiene hijo.



El diagrama muestra un robot de dos articulaciones, donde cada transición entre eslabones se modela mediante transformaciones geométricas compuestas (rotación + traslación). La posición final del efector resulta de la aplicación secuencial de estas transformaciones desde la base.

Finalmente, se resume la posición del efector final como una composición de transformaciones desde la base hasta el último eslabón.

Se puede resumir un robot como:

$${}^0\xi_N = {}^0\xi_1(q_0) \oplus {}^1\xi_2(q_1) \oplus \cdots \oplus {}^{N-1}\xi_N(q_{N-1})$$

$${}^0\xi_N = \mathcal{K}(q)$$

$\mathcal{K}(q)$

Donde $\mathcal{K}(q)$ representa la posición y orientación del efector final como función de las variables articulares q .

Creamos la cadena de eslabones:

```
link1 = Link2(ET2.R(), name="link1");
link2 = Link2(ET2.tx(1) * ET2.R(), name="link2", parent=link1);
link3 = Link2(ET2.tx(1), name="link3", parent=link2);
```

✓ 0.0s

```
robot = ERobot2([link1, link2, link3], name="my robot")
```

✓ 0.0s

ERobot: my robot, 2 joints (RR)

link	link	joint	parent	ETS: parent to link
0	link1	0	BASE	$R(q_0)$
1	link2	1	link1	$tx(1) \oplus R(q_1)$
2	@link3		link2	$tx(1)$

```
robot.fkine(np.deg2rad([30, 40])).println()
```

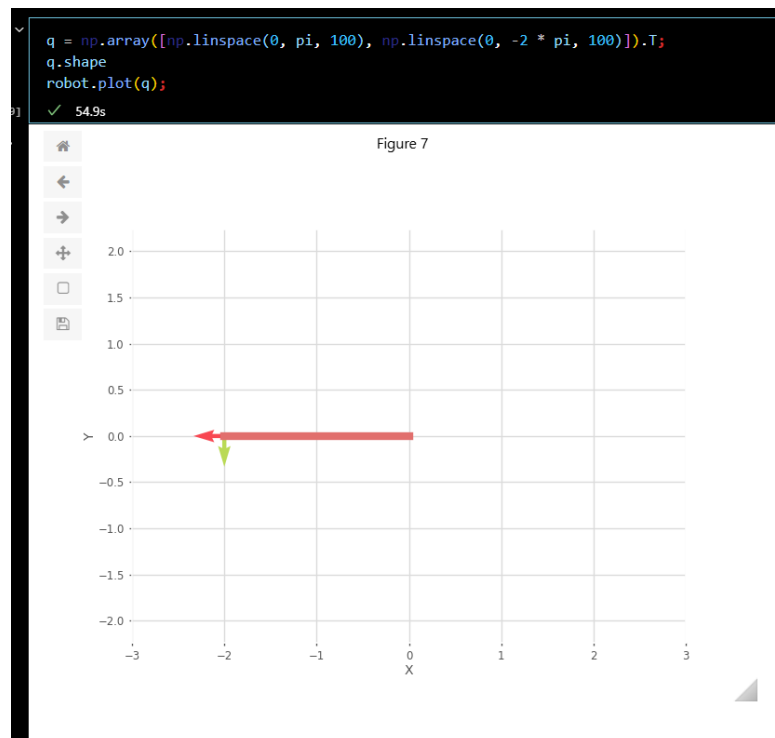
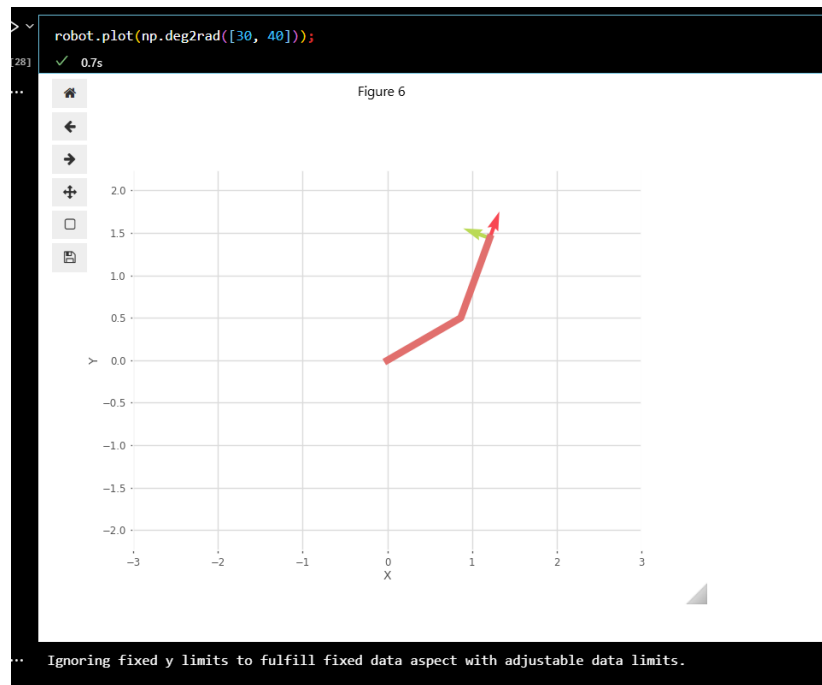
✓ 0.0s

t = 1.21, 1.44; 70°

Cada parte de la cadena puede tener máximo un parámetro variable. La primera columna enumera los eslabones, la segunda indica su nombre (el @ indica que es un efector), la tercera columna enumera las articulaciones (piezas móviles que tienen un parámetro y representan un DoF), la cuarta columna indica quien es su eslabón padre (punto donde estaría el marco de referencia antes de aplicar las transformaciones del eslabón) y la última columna indica el ETS (Secuencia de Transformación Esencial).

Animación y visualización:

- Se puede realizar una **animación** utilizando el método `.plot()` para visualizar el movimiento del robot. En este caso, si la primera articulación gira **180°** y la segunda realiza un **giro completo**, el efector se desplaza de **2 a -2 en el eje X**, mientras que se mantiene en **0 en el eje Y**.



Tenemos formas de ir por cada eslavón:

```
[30] robot[1]
✓ 0.0s

... Link2("link2", tx(1)  $\oplus$  R(q), parent="link1")

[31] robot["link2"]
✓ 0.0s

... Link2("link2", tx(1)  $\oplus$  R(q), parent="link1")
```

De igual manera, buscamos mostrar efectores, u otros atributos de esta manera:

```
✓ robot.ee_links
[2] ✓ 0.0s

+ [Link2("link3", tx(1), parent="link2")]
```

```
link2.parent
```

```
[33] ✓ 0.0s
```

```
... Link2("link1", R(q))
```

```
link2.children
```

```
[34] ✓ 0.0s
```

```
... [Link2("link3", tx(1), parent="li
```

```
link2.jindex
```

```
[35] ✓ 0.0s
```

```
... 1
```

```
link2.isrevolute  
link2.isprismatic
```

```
[36] ✓ 0.0s
```

```
... False
```

```
print(link2.qlim)
```

```
[37] ✓ 0.0s
```

```
... None
```

```
link2.A(pi / 6)
```

```
[38] ✓ 0.0s
```

```
... 

|       |       |   |
|-------|-------|---|
| 0.866 | -0.5  | 1 |
| 0.5   | 0.866 | 0 |
| 0     | 0     | 1 |


```

```
▶ ✓ link2.ets
```

```
[39] ✓ 0.0s
```

```
...  $tx(1) \oplus R(q)$ 
```


Caso 3D:

Se puede crear todo el robot y el programa separa automáticamente por eslabones:

```
a1 = 1; a2 = 1;
robot6 = ERobot(ET.Rz() * ET.Ry() * ET.tz(a1) * ET.Ry() \
               * ET.tz(a2) * ET.Rz() * ET.Ry() * ET.Rz())
✓ 0.0s
```

ERobot: , 6 joints (RRRRRR)

link	link	joint	parent	ETS: parent to link
0	link0	0	BASE	Rz(q0)
1	link1	1	link0	Ry(q1)
2	link2	2	link1	tz(1) ⊕ Ry(q2)
3	link3	3	link2	tz(1) ⊕ Rz(q3)
4	link4	4	link3	Ry(q4)
5	@link5	5	link4	Rz(q5)

El toolbox incluye robots construídos previamente, por lo que, creando el archivo panda se obtendrá lo siguiente:

```
models.list(type="ETS")
✓ 0.0s
```

class	name	manufacturer	type	DoF	dims	structure	dynamics	geometry	keywords
Panda	Panda	Franka Emika	ETS	7	3d	RRRRRRR			mobile
Frankie	Frankie	Franka Emika, Omron	ETS	9	3d	RPrrrrrrrr			
Puma560	Puma560	Unimation	ETS	6	3d	RRRRRR			
Planar_Y	Planar-Y		ETS	6	3d	RRRRRR			
GenericSeven	Generic Seven	Jesse's Imagination	ETS	7	3d	RRRRRRR			
XYPanda	XYPanda	Franka Emika	ETS	9	3d	PPrrrrrrrr			

```
panda = models.ETS.Panda()
```

Este objeto tipo panda, tiene un atributo para los ángulos, al cual se accede por diccionario y se puede saber dónde está el efector con el mismo comando:

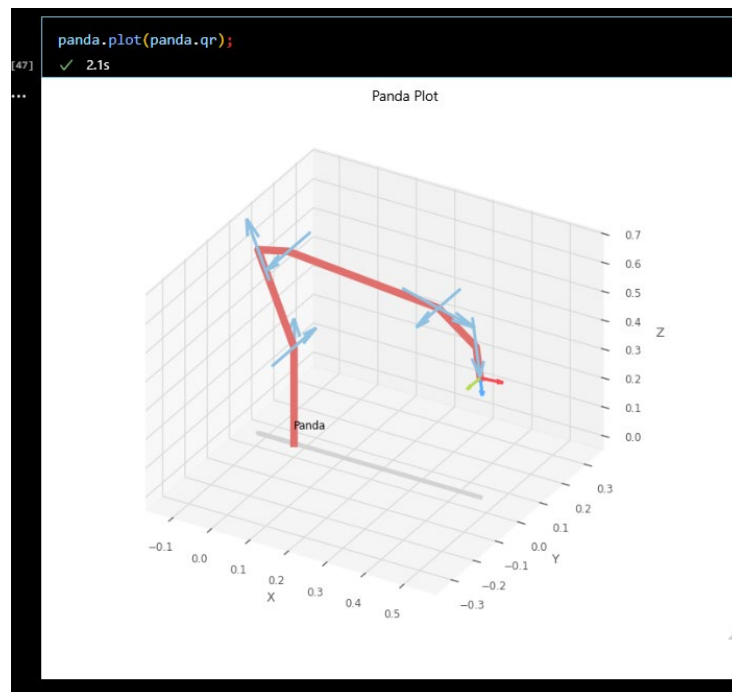
```
> panda.addconfiguration("foo", [1, 2, 3, 4, 5, 6, 7])
44] ✓ 0.0s

panda.configs["foo"];
panda.configs["qz"];
45] ✓ 0.0s

panda.fkine(panda.qr).printline()
46] ✓ 0.0s

.. t = 0.484, 0, 0.413; rpy/zyx = 180°, -5.73°, 0°
```

Finalmente, graficamos:



Finalmente, vemos como en T se guarda la matriz de cada articulación respecto a la base y a cada articulación:

```
T = panda.fkine_all(panda.qr);
len(T)
✓ 0.0s
```

9

0:

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

1:

1	0	0	0
0	1	0	0
0	0	1	0.333
0	0	0	1

2:

0.9553	0.2955	0	0
0	0	1	0
0.2955	-0.9553	0	0.333
0	0	0	1

3:

0.9553	0	-0.2955	-0.09338
0	1	0	0
0.2955	0	0.9553	0.6349
0	0	0	1

```
T[1].printline()
[49] ✓ 0.0s
```

... t = 0, 0, 0.333; rpy/zyx = 0°, 0°, 0°

Bases y Herramientas:

```
50] panda.base = SE3.Tz(3) * SE3.Rx(pi); # robot is 3m up and hanging down
panda.tool = SE3.Tz(0.15); # tool is 150mm long in z-direction
✓ 0.0s

51] panda.fkine(panda.qr, tool=SE3.Trans(0.05, 0.02, 0.20) * SE3.Rz(np.deg2rad(45)));
✓ 0.0s
```

7.2 Cinemática Inversa.

El procedimiento para determinar los parámetros del robot con el objetivo de alcanzar una posición deseada se centra en la cinemática inversa de un robot plano con dos grados de libertad (2D). Esta técnica implica calcular los ángulos necesarios en las articulaciones para que el efector final del robot llegue a la posición deseada en el espacio. Para ello, se emplea una solución cerrada, que es un enfoque analítico que permite despejar directamente las variables necesarias mediante ecuaciones simbólicas.

7.2.1.1 Solución Cerrada 2D.

Parte analítica, matemática, involucra despejar ecuaciones.

```
import sympy
a1, a2 = sympy.symbols("a1 a2")
e = ET2.R() * ET2.tx(a1) * ET2.R() * ET2.tx(a2);
1 ✓ 0.0s

q0, q1 = sympy.symbols("q0 q1")
1 ✓ 0.0s

TE = e.fkine([q0, q1])
x_fk, y_fk = TE.t;
1 ✓ 1.7s

x, y = sympy.symbols("x y")
1 ✓ 0.0s
```

Primero se crea el robot con sus variables simbólicas y simbólicamente sacamos la matriz de transformación homogénea. Guardamos la condición final de cada caso con la transpuesta.

```

[34] eq1 = (x_fk**2 + y_fk**2 - x**2 - y**2).trigsimp()
✓ 0.2s

...  $a_1^2 + 2a_1a_2 \cos(q_1) + a_2^2 - x^2 - y^2$ 

▷ ✓
[35] q1_sol = sympy.solve(eq1, q1)
eq0 = tuple(map(sympy.expand_trig, [x_fk - x, y_fk - y]))
q0_sol = sympy.solve(eq0, [sympy.sin(q0), sympy.cos(q0)]);
✓ 0.3s

[36] sympy.atan2(q0_sol[sympy.sin(q0)], q0_sol[sympy.cos(q0)]).simplify()
✓ 0.4s

...  $\text{atan}_2 \left( \frac{a_1 y - a_2 x \sin(q_1) + a_2 y \cos(q_1)}{a_1^2 + 2a_1a_2 \cos(q_1) + a_2^2}, \frac{a_1 x + a_2 x \cos(q_1) + a_2 y \sin(q_1)}{a_1^2 + 2a_1a_2 \cos(q_1) + a_2^2} \right)$ 

```

Se definen las longitudes de los eslabones y las transformaciones homogéneas correspondientes, que incluyen rotaciones y traslaciones. A partir de este modelo, se calcula la posición del efector final en función de los ángulos articulares, lo que permite plantear ecuaciones que igualen esa posición con una coordenada deseada.

Luego, se resuelven estas ecuaciones de manera analítica para obtener los ángulos que permiten alcanzar la posición objetivo. Primero, se despeja uno de los ángulos utilizando una ecuación escalar basada en las distancias, aplicando identidades trigonométricas y simplificaciones. Después, se despeja el otro ángulo separando las componentes de las coordenadas, y utilizando las funciones trigonométricas correspondientes, que se combinan mediante una función específica para obtener el ángulo de forma completa y sin ambigüedades. Este proceso permite determinar todas las posibles configuraciones articulares para una posición dada.

7.2.1.1 Solución Numérica 2D.

En esta sección se presenta una solución numérica para la cinemática inversa de un robot plano de dos grados de libertad. A diferencia del enfoque analítico previamente descrito, este método trata el problema como una minimización, donde se busca encontrar los ángulos articulares que reduzcan al mínimo la diferencia entre la posición deseada del efector final y la posición actual obtenida por

cinemática directa. En otras palabras, el objetivo es minimizar la discrepancia entre la pose actual y la pose deseada.

Para ello, primero se define el robot y se establece la posición a la que se quiere llevar, luego se crea una función de minimización y se aplica el método correspondiente. Se define simbólicamente el robot, se fija la posición objetivo, y se construye una función de error que mide la distancia euclidiana entre la posición alcanzada y la deseada.

```
[37] e = ET2.R() * ET2.tx(1) * ET2.R() * ET2.tx(1);
✓ 0.0s

[38] pstar = np.array([0.6, 0.7]); # desired position
E = lambda q: np.linalg.norm(e.fkine(q).t - pstar);
✓ 0.0s

> ✓
[39] sol = optimize.minimize(E, [0, 0]);
✓ 0.0s

[40] sol.x
✓ 0.0s
... array([ -0.2295,  2.183])

[41] e.fkine(sol.x).printline()
✓ 0.0s
... t = 0.6, 0.7; 112°
```

A continuación, se emplea el método `optimize.minimize()` para resolver el problema de optimización, comenzando desde una condición inicial de `[0, 0]`. El resultado de este proceso es un conjunto de valores para los ángulos articulares que aproximan al robot a la posición deseada con alta precisión. Finalmente, se verifica la posición alcanzada con estos valores mediante la cinemática directa. Es importante señalar que pueden existir múltiples soluciones válidas, pero la solución obtenida dependerá directamente de las condiciones iniciales utilizadas durante el proceso de optimización.

7.2.2.1 Solución Cerrada 3D.

Algunos modelos de robots cuentan con un método para resolver la cinemática inversa. A partir de una configuración inicial de ángulos, se puede obtener la pose final del efector

```
[42] puma = models.DH.Puma560();
✓ 0.0s

[43] puma.qn
✓ 0.0s
... array([ 0, 0.7854, 3.142, 0, 0.7854, 0])
```

En el caso de este robot, que posee una muñeca esférica (es decir, un brazo robótico con una muñeca que puede rotar en los tres ejes), es posible realizar el análisis de la cinemática inversa para determinar la pose final.

```
[44] T = puma.fkine(puma.qn);
T.println()
✓ 0.0s
... t = 0.596, -0.15, 0.657; rpy/zyx = 0°, 90°, 0°
```

Como este robot tiene una muñeca esférica (es un brazo robótico con muñeca con rotación en los tres ejes). Puede hacerse la analítica de la cinemática inversa a la pose final.

```
sol = puma.ikine_a(T)
✓ 0.0s
IKSolution(q=array([ 2.649, 2.356, 0.09396, -0.609, -0.9743, -2.768]), success=True, iterations=0, searches=0, residual=0.0, reason='')

sol.q
✓ 0.0s
array([ 2.649, 2.356, 0.09396, -0.609, -0.9743, -2.768])
```

Aunque dan una solución diferente se puede comprobar.

```
puma.fkine(sol.q).println()
[47] ✓ 0.0s
... t = 0.596, -0.15, 0.657; rpy/zyx = 0°, 90°, 0°
```

Aunque las soluciones obtenidas pueden ser diferentes, estas pueden ser verificadas. La diferencia en las soluciones se debe a las condiciones iniciales del robot, las cuales se pueden especificar como parámetros del método. Estos parámetros incluyen:

- Mano izquierda o derecha: "l" o "r".
- Codo arriba o abajo: "u" o "d".
- Muñeca volteada o no: "f" o "n".

Estos parámetros nos proporcionan los mismos ángulos que al inicio.

```
sol = puma.ikine_a(T, "r");
sol.q
[48] ✓ 0.0s
... array([ 0, 0.7854, -3.142, -3.142, -0.7854, 3.142])
```

Lo que nos da los mismos ángulos que al inicio.

Sin embargo, pueden presentarse situaciones en las que el robot no pueda alcanzar una posición deseada debido a limitaciones físicas o colisiones. Por ejemplo, si el brazo no es lo suficientemente largo, el robot no podrá llegar al destino.

```
puma.ikine_a(SE3.Tx(3))
[49] ✓ 0.0s
... IKSolution(q=None, success=False, iterations=0, searches=0, residual=0.0, reason='Out of reach')
```

En cuanto a la muñeca con los tres ejes de rotación, presenta una singularidad conocida como "gimbal lock" cuando el segundo eje es cero y los ejes primero y tercero se alinean. En este caso, el resultado puede distribuirse arbitrariamente entre los ejes (por ejemplo, $0.1 + 0.2 = 0.3$).


```

[50] q = [0, pi/4, pi, 0.1, 0, 0.2];
✓ 0.0s

[51] puma.ikine_a(puma.fkine(q), "ru").q
✓ 0.0s

... array([ 0, 0.7854, -3.142, 0, 0, 0.3])

```

yyyy

7.2.2.2 Solución Numérica 3D.

Si no tiene solución analítica o no tiene la muñeca esférica toca por este lado:

```

> T = puma.fkine(puma.qn);
  T.println("rpy/xyz")
  sol = puma.ikine_LM(T)
[53] ✓ 0.0s
... t = 0.596, -0.15, 0.657; rpy/xyz = 0°, 90°, 0°

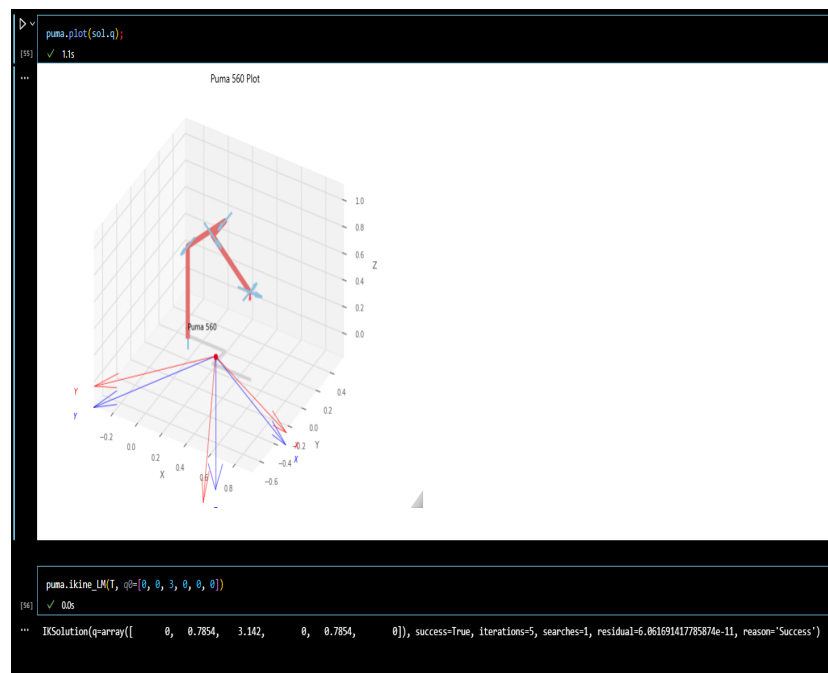
... IKSolution(q=array([ 0, 0.7854, 3.142, -3.142, -0.7854, 3.142]), success=True, iterations=17, searches=1, residual=1.322016349997858e-09, reason='Success')

puma.qn
[53] ✓ 0.0s
... array([ 0, 0.7854, 3.142, 0, 0.7854, 0])

puma.fkine(sol.q).println("rpy/xyz")
[54] ✓ 0.0s
... t = 0.596, -0.15, 0.657; rpy/xyz = 0°, 90°, 0°

```

yy



t

7.2.3 Manipulador Subactuado

Un manipulador subactuado (underactuated) es aquel que tiene menos de seis grados de libertad (juntas o actuadores). Esto implica que:

- No puede alcanzar todas las posiciones y orientaciones posibles del efector final en el espacio tridimensional.
- Está limitado a un subconjunto específico del espacio de tareas.

Un ejemplo de esto son los robots SCARA, que están diseñados principalmente para tareas de pick and place. Estos robots no pueden lograr que la pose del efector final tenga una rotación en la que no esté paralela a la vertical.

En este caso, se importa el modelo Cobra600 y se intentará alcanzar una pose “torcida”.

```
cobra = models.DH.Cobra600()
✓ 0.0s
```

DHRobot: Cobra600 (by Omron), 4 joints (RRPR), dynamics, standard DH parameters

θ_j	d_j	a_j	α_j	q^-	q^+
q1	0.387	0.325	0.0°	-50.0°	50.0°
q2	0	0.275	180.0°	-88.0°	88.0°
0.0°	q3	0	0.0°	0.0	0.21
q4	0	0	0.0°	-180.0°	180.0°

name	q0	q1	q2	q3
qz	0°	0°	0	0°
qr	0°	0°	0	0°

Como se puede ver dice que no llegó a una solución y paró de intentarlo.

Para estos casos podemos relajar las condiciones finales de una parte de nuestra pose para así encontrar una pose que cumpla lo mejor posible.

```
cobra.fkine(sol.q).println("rpy/xyz")
✓ 0.0s

t = 0.4, -0.3, 0.2; rpy/xyz = 30°, 0°, 180°
```

Vemos que llegamos a una buena aproximación a lo deseado.

7.2.4 Manipulador Sobreactuado

Problemas como los límites de las juntas, la auto colisión y las singularidades implican que no todas las poses dentro del espacio accesible del robot son alcanzables. Una forma de superar este desafío es agregar más juntas al robot.

Esto genera un número infinito de soluciones posibles para las coordenadas articulares, pero para obtener una solución única, es necesario introducir restricciones. Una restricción común es la de la norma mínima, que selecciona una solución en la que la norma del vector de coordenadas articulares tiene la menor magnitud posible.

Volviendo al ejemplo del robot Panda, que tiene siete articulaciones (por lo tanto, está sobreactuado), se desea alcanzar una pose en un lugar específico utilizando la notación de dos vectores para las garras (vector a-o), con su vector de aproximación apuntando hacia abajo.

El método utilizado busca la norma más pequeña para el vector de las articulaciones, lo que minimiza el esfuerzo articular total al generar la menor magnitud de movimiento articular posible.

```
panda = models.ETS.Panda();
✓ 0.0s

TE = SE3.Trans(0.7, 0.2, 0.1) * SE3.OA((0, 1, 0), (0, 0, -1));
✓ 0.0s

sol = panda.ikine_lm(TE)
✓ 0.0s

IKSolution(q=array([ 0.6647,  2.306, -2.618, -1.401,  2.693,  1.024, -2.306]), success=True, iterations=40, searches=2, residual=2.7928395224299336e-07, reason='Success')

panda.fkine(sol.q).println("angvec")
✓ 0.0s

t = 0.7, 0.2, 0.1; angvec = (180° | 0, 1, 0)
```

7.3 Trayectorias

En esta sección se trata el tema de las trayectorias en robótica, centrándose en el movimiento suave de un brazo robótico desde una pose inicial hasta una pose final. Se hace una distinción entre mover el brazo a lo largo de trayectorias rectas en el espacio de configuración (espacio de las articulaciones) o en el espacio de tareas (el espacio cartesiano del efector).

En términos más específicos, se describe el procedimiento para mover un brazo robótico suavemente de una pose a otra. Este movimiento puede realizarse siguiendo trayectorias rectas, ya sea en el espacio de configuración o en el espacio de tareas.

7.3.1.1 Joint-Space Motion.

Donde el trayecto se define directamente en los ángulos articulares del robot, lo cual simplifica el control, pero no necesariamente garantiza un movimiento recto del efector en el espacio. Se parte de definir las poses de inicio y fin; y cuáles son los parámetros que pueden llegar a esas configuraciones (caso right-handed elbow-up configuración /ru).

```
puma = models.DH.Puma560();
```

✓ 0.0s

7.3.1 Joint-Space Motion

```
TE1 = SE3.Trans(0.4, -0.2, 0) * SE3.Rx(3);
```

```
TE2 = SE3.Trans(0.4, 0.2, 0) * SE3.Rx(1);
```

✓ 0.0s

```
sol1 = puma.ikine_a(TE1, "ru");
```

```
sol2 = puma.ikine_a(TE2, "ru");
```

✓ 0.0s

Se definen dos poses (TE1 y TE2) mediante transformaciones homogéneas del tipo SE(3), que incluyen tanto traslación como rotación. Luego, se aplica la cinemática inversa al robot PUMA para calcular los ángulos articulares correspondientes a cada pose, utilizando la función *ikine_a* y una configuración específica ("ru": brazo derecho con el codo arriba). El resultado arroja dos soluciones válidas, cada una con seis ángulos articulares, lo que confirma que el robot puede alcanzar ambas poses y que es posible planificar una trayectoria entre ellas en el espacio articular.

Se desea una trayectoria que dure 2 segundos, con pasos de 0.2 segundos.

```
t = np.arange(0, 2, 0.02);  
✓ 0.0s
```

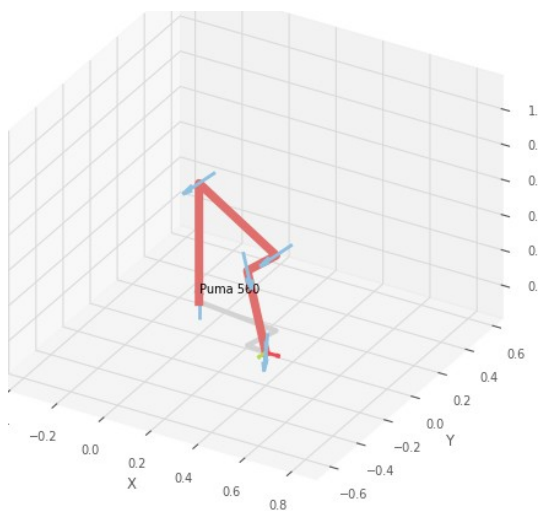
Y con eso se pueden crear las trayectorias, ya sea con un polinomio quintico o una velocidad trapezoidal.

```
traj = mtraj(quintic, sol1.q, sol2.q, t);  
✓ 0.0s  
  
traj = mtraj(trapezoidal, sol1.q, sol2.q, t);  
✓ 0.0s  
  
traj.q.shape  
✓ 0.0s  
  
(100, 6)
```

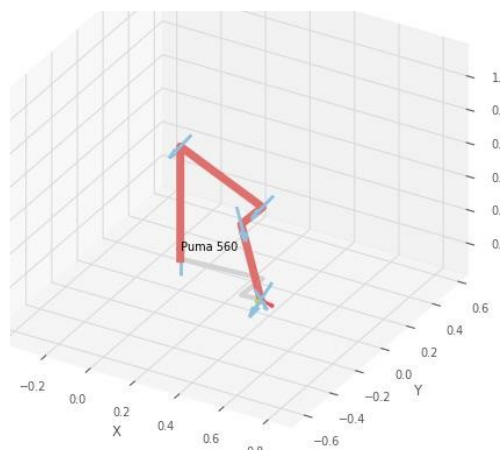
El tamaño de ambos es de 100 por 6.

También se puede usar esta que está más optimizada para caso 3D

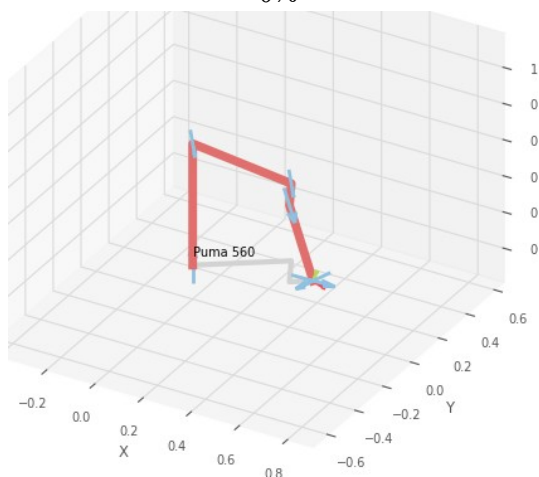
```
traj = jtraj(sol1.q, sol2.q, t)  
✓ 0.0s  
  
Trajectory created by jtraj: 100 time steps x 6 axes
```



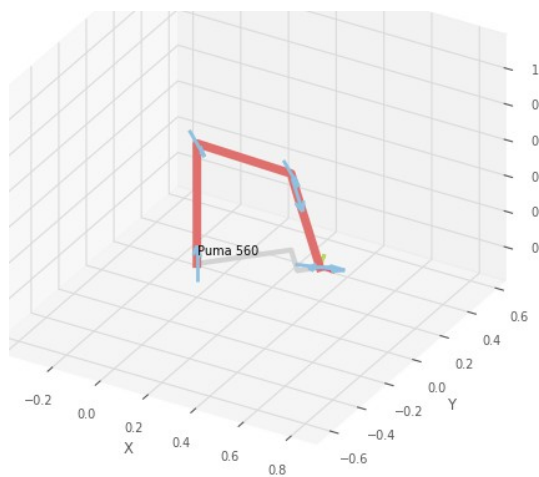
0%



33%

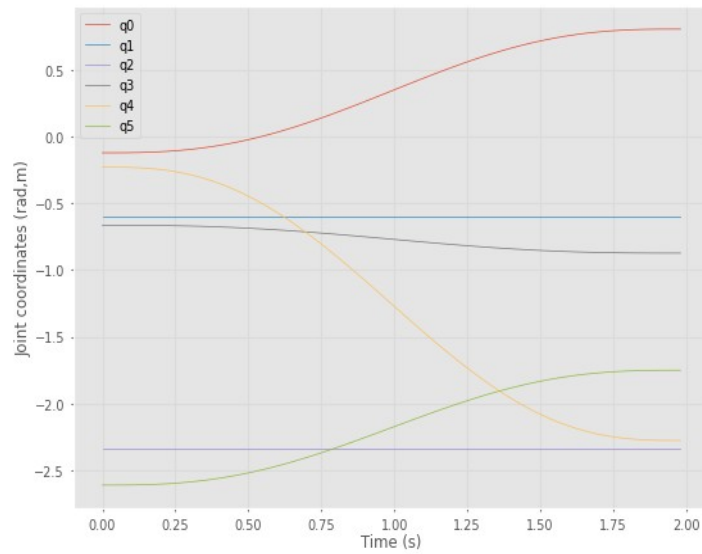


66%



100%

Se puede ver cómo cambia el efector al final.

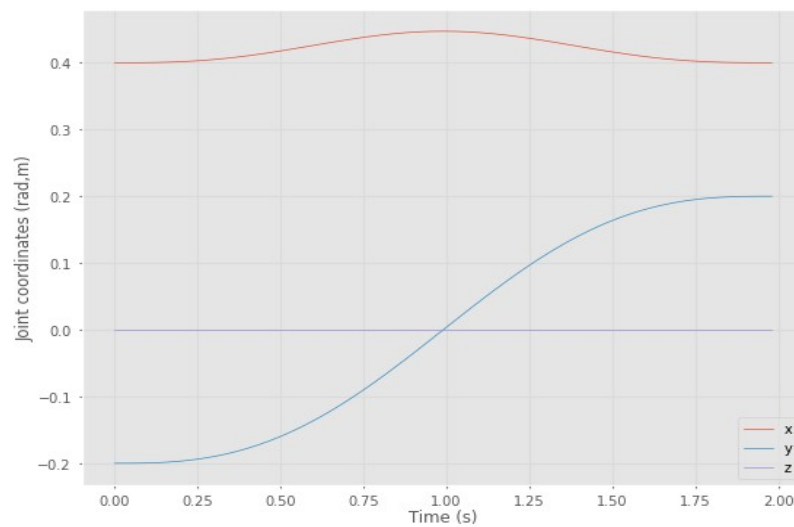


Podemos ver lo relacionado a la traslación de tal forma que:

```

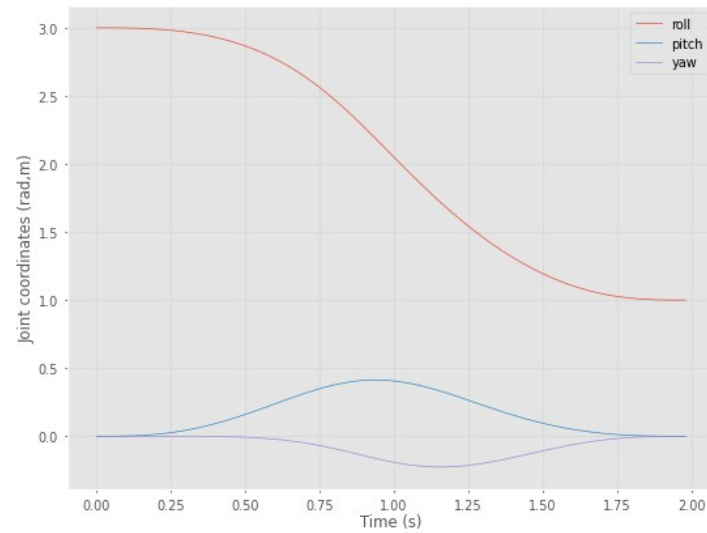
..
T = puma.fkine(traj.q);
len(T)
77] ✓ 0.0s
.. 100

```



Y para los relacionados con la rotación:

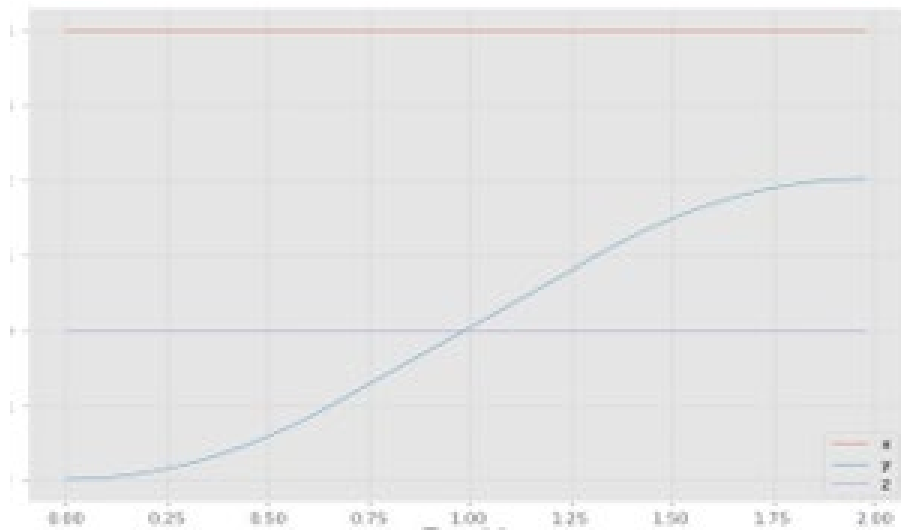
```
[79] xplot(t, T.t, labels="x y z");  
✓ 0.1s  
...
```



7.3.2 Cartesian Motion

Aquí, procedemos a trabajar con las matrices de transformación homogéneas.

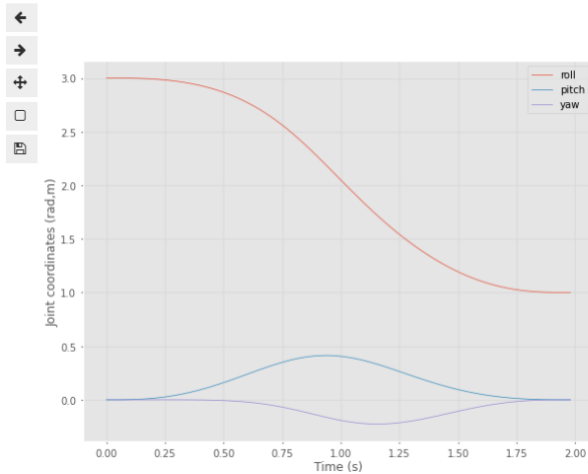
```
81] Ts = ctraj(TE1, TE2, t);  
✓ 0.0s  
  
82] xplot(t, Ts.t, labels="x y z");  
✓ 0.2s  
..
```

```
xplot(t, T.rpy("xyz"), labels="roll pitch yaw");
```

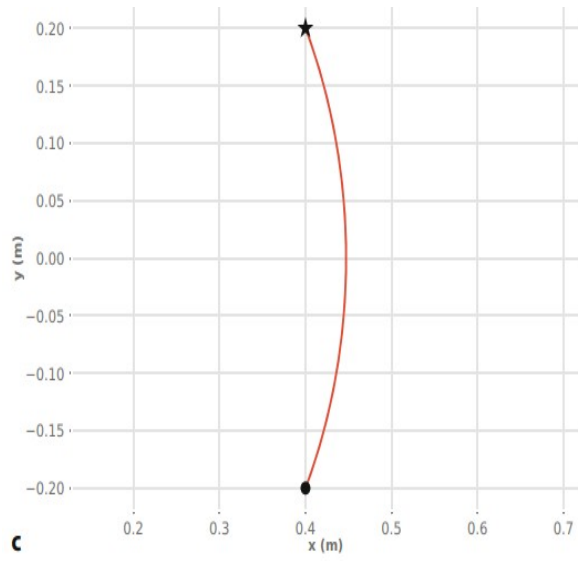
✓ 0.1s

Figure 11

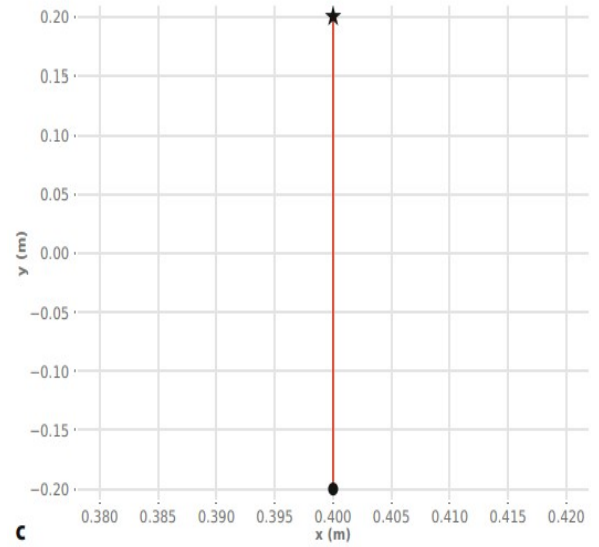


(x, y) = (2.035, 1.332)

La diferencia entre una y otra se ve a continuación en el plano cartesiano x-y:



J_{traj}



C_{traj}

7.3.4 Movimiento a través de una singularidad

Se observa cómo manejar la representación de las gráficas respecto a las singularidades:

```

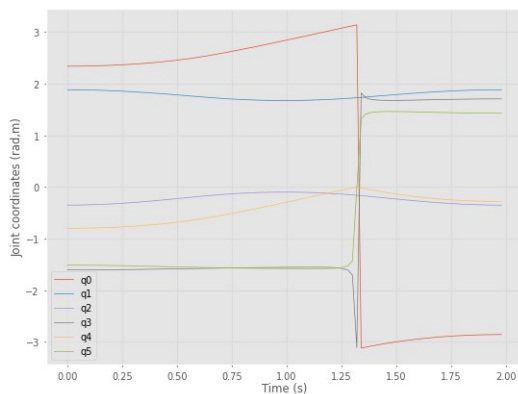
86] TE1 = SE3.Trans(0.5, -0.3, 1.12) * SE3.OA((0, 1, 0), (1, 0, 0));
    TE2 = SE3.Trans(0.5, 0.3, 1.12) * SE3.OA((0, 1, 0), (1, 0, 0));
    ✓ 0.0s

87] Ts = ctraj(TE1, TE2, t);
    ✓ 0.0s

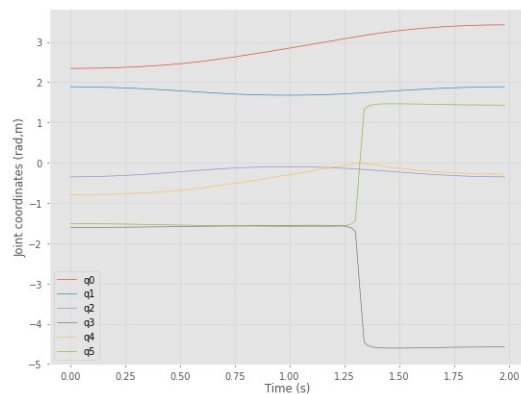
88] sol = puma.ikine_a(Ts, "lu");
    ✓ 0.0s

89] xplot(t, sol.q, unwrap=True);
    ✓ 0.2s

```



Falso

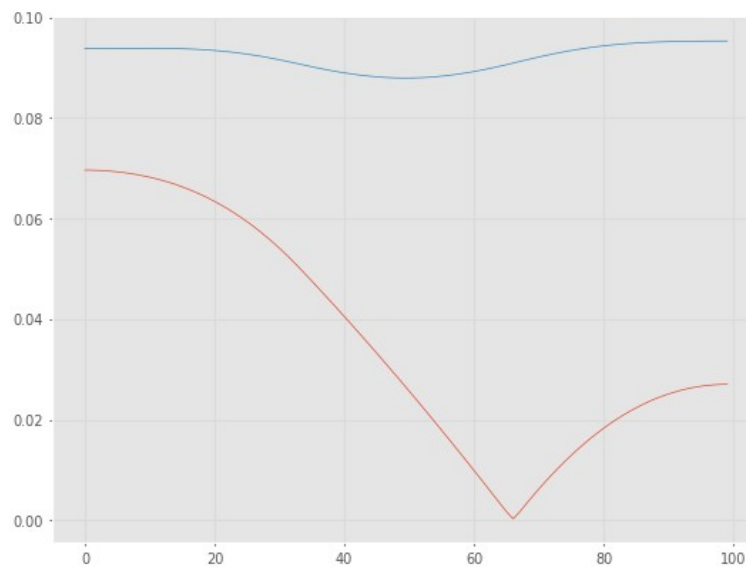


Verdadero

En 1.3 se llega cerca de una singularidad, por lo tanto, se ve un cambio rápido.

Manipulabilidad:

- Es una medida de cuán fácilmente puede moverse o rotar el efector final del robot.
- Se calcula utilizando la matriz Jacobiana del robot.
- A un tiempo aproximado de 1.3 segundos, la manipulabilidad cae casi a cero, lo que indica que el robot pierde destreza.
- También se observa que el ángulo q_3 cambia muy poco, lo cual es una estrategia de los métodos numéricos para evitar las singularidades.



7.3.5 Configuración del Cambio

Cuando dos configuraciones tienen la misma pose del efector, no es posible interpolar en el espacio cartesiano.

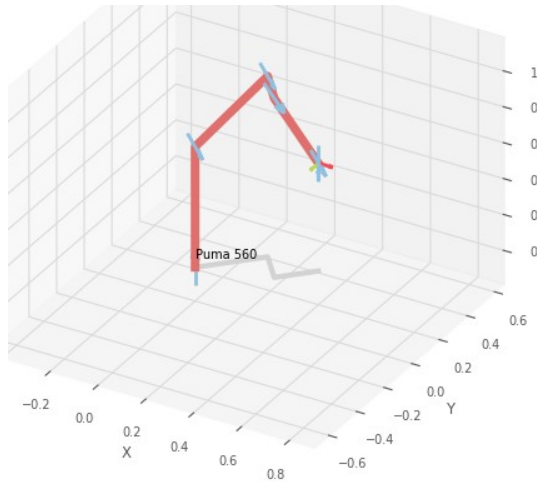
En estos casos, hay que planear el movimiento considerando directamente las articulaciones.

```
[91] TE = SE3.Trans(0.4, 0.2, 0.6) * SE3.Rx(pi);
      ✓ 0.0s

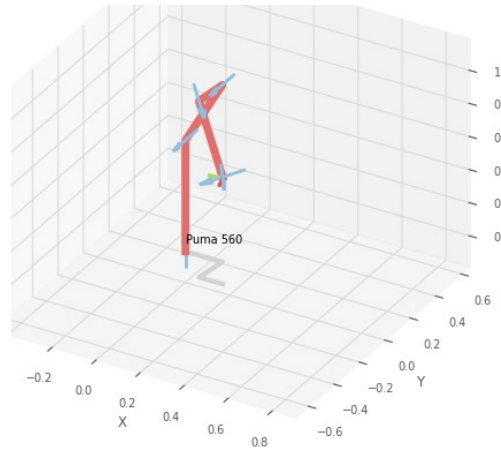
[92] sol_r = puma.ikine_a(TE, "ru");
      sol_l = puma.ikine_a(TE, "lu");
      ✓ 0.0s

[93] traj = jtraj(sol_r.q, sol_l.q, t);
      ✓ 0.0s

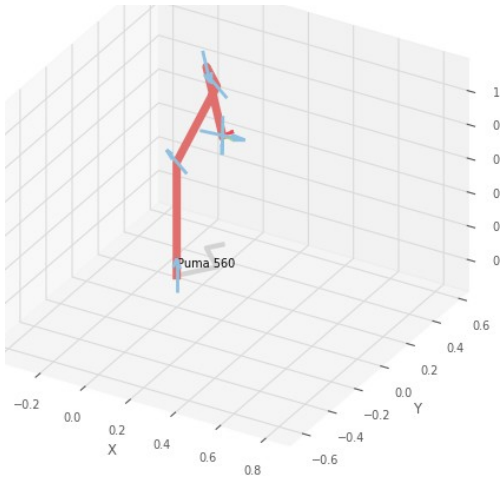
[94] puma.plot(traj.q);
      ✓ 0.0s
```



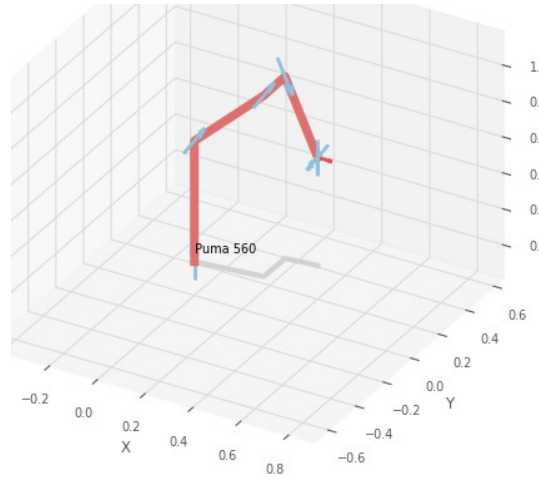
0%



33%



66%



100%

Vemos como se está realizando una rotación en el eje -z, posteriormente se crea una trayectoria en el espacio articular que conecta dos configuraciones del robot, las cuales, aunque presentan la misma pose del efector final, tienen posiciones articulares diferentes.

sol_r.q: es la configuración inicial (por ejemplo, "mano derecha").

sol_l.q: es la configuración final (por ejemplo, "mano izquierda").

Conclusiones

- Las simulaciones realizadas demostraron que la combinación de cinemática inversa y el uso de sensores ultrasónicos en robots permite una navegación reactiva eficiente, ajustando la trayectoria del robot en tiempo real. Esta capacidad es esencial para operar en entornos dinámicos y cambiantes, donde los obstáculos son detectados y evitados sin planificación previa, asegurando una ejecución óptima en espacios complejos.
- La integración de la cinemática inversa con el sensor ultrasónico mejora significativamente la precisión del robot al navegar, permitiéndole ajustar sus movimientos de manera continua. Este enfoque no solo permite sortear obstáculos, sino también adaptar su orientación y posición final, logrando un control más preciso sobre su trayectoria en el espacio
- La combinación de articulaciones rotacionales y prismáticas en el diseño de robots otorga un alto grado de flexibilidad y capacidad para realizar tareas complejas. Este diseño mejora la precisión en los movimientos del robot, lo que es fundamental para operaciones que requieren un control exacto, como la manipulación y el ensamblaje en espacios 2D.

Referencias

- Corke, P. Robotics, Vision and Control v2 (RVC)
- notebook : chap7.ipynb
- Documentación Robotics Toolbox
- 7_CinemáticaDelManipulador.pdf (Presentación MOODLE)
- Git hub RVC v2